# RL I

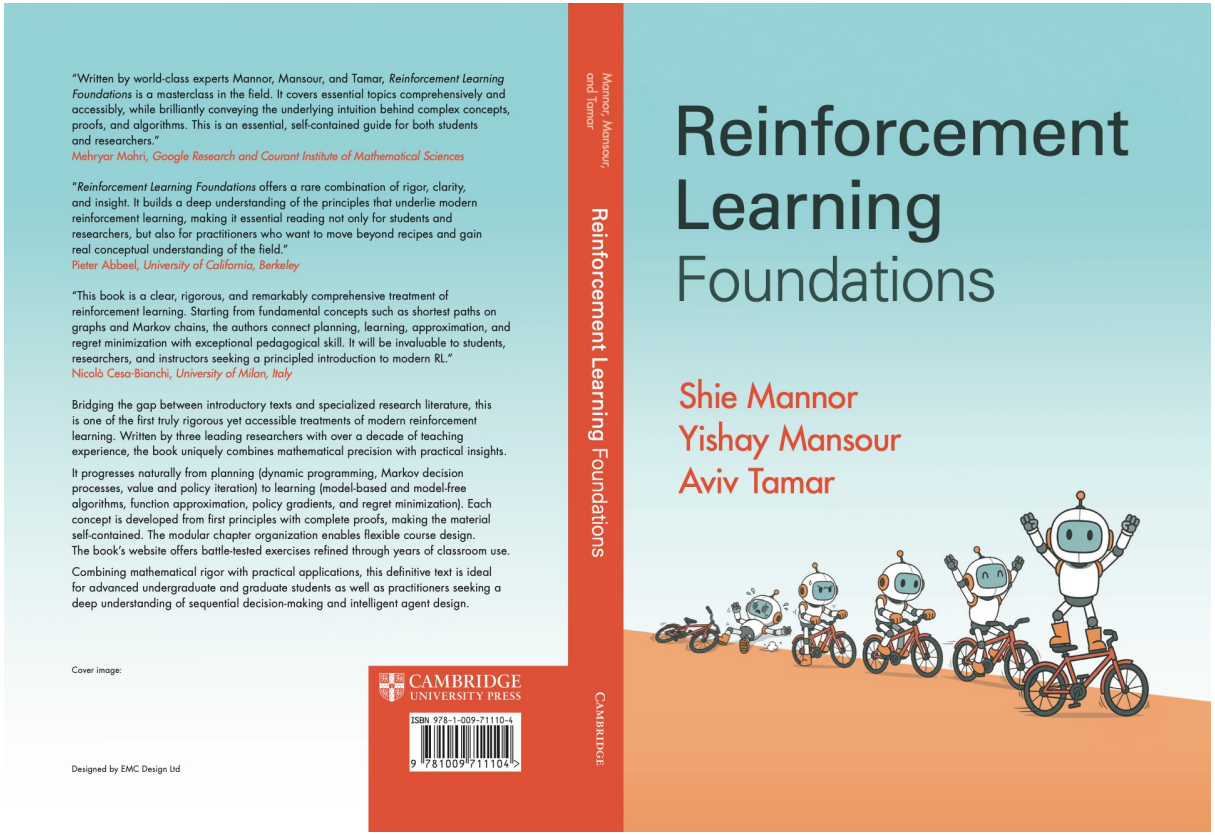
Shie Mannor

NVIDIA Research
Technion

# Our book

- https://sites.google.com/view/rlfoundations/home

- Coming this summer from CUP



# Plan for today

1. Model-based RL
2. Q-Learning, SARSA, Monte-Carlo
3. TD Learning

As this is RL#1 everything is in finite spaces.

# MDP - computing optimal policy is easy (!)

1. Linear Programming

2. Value Iteration method.

$$V^{t+1}(s) \leftarrow \max_a \{R(s,a) + \gamma \sum_{s'} \delta(s,a,s') V^t(s')\}$$

3. Policy Iteration method.

$$\pi_i(s) = \arg\max_a \{Q^{\pi_{i-1}}(s,a)\}$$

# Learning Algorithms

Given access only to actions perform:

1. policy evaluation.
2. control - find optimal policy.

Two approaches:

1. Model based.
2. Model free.

# Plan for today

1. Model based RL

2. Q-Learning, SARSA, Monte-Carlo

3. TD Learning for policy evaluation

As this is RL#1 everything is in finite spaces.

# Learning - Model Based

Estimate the model from the observation.
(Both transition probability and rewards.)

Use the estimated model as a true model, and find a near optimal policy. AKA the "certainty equivalence" principle

If we have a "good" estimated model, we should have a "good" estimation.

# Learning - Model Based: off policy

- Let the policy run for a “long” time.
  - what is “long” ?!
  - Assuming some “exploration”
- Build an “observed model”:
  - Transition probabilities
  - Rewards
    - Both are independent!
- Use the “observed model” learn optimal policy.

# Learning - Model Based – Certainty Equivalence

off-policy algorithm

- Observe a trajectory generated by a policy π
  - off-policy: no need to control
- For every $s, s' \in S$ and $a \in A$:
  - $\mathrm{p}'(s, a, s') = \#(s, a, s') \,/\, \#(s, a, \cdot)$
  - $R'(s, a) = AVG(r \mid s, a)$
- Find an optimal policy for $(S, a, p', R')$

# Learning - Model Based

- Claim: if the model is “accurate” we will compute a near-optimal policy.
- Hidden assumption:
  - Each (s,a,·) is sampled many times.
  - This is the “responsibility” of the policy π
    - Off-policy
- Simple question: How many samples we need for each (s,a,·)?

# Toolkit: Off-Policy Model-based learning

- Input
  - Sequence of $(s, a, r, s')$
    - $r \sim R(s, a)$
    - $s' \sim p(\cdot\,|s, a)$
- Output
  - Complete MDP model
    - $r(s, a)$ and $p(s'|s, a)$

# Mean Estimation

- Assume we have a r.v.
  - $R \in [0, R_{max}]$
- Goal:
  - Estimate $E[R]$
- Tool: sampling
  - Get samples $R_1, \ldots, R_m$
  - Estimate using average: $\hat{r} = \frac{1}{m} \sum_{i=1}^{m} R_i$

- How good is the mean?
  - $\hat{r} \rightarrow_{m \rightarrow \infty} E[R]$
  - Law of large numbers
- How many samples do we need?
  - How to choose *m*?
  - Finite convergence bounds

# Concentration bounds

- Let $X_i$ be i.i.d. r.v.
  - $X_i \in [0,1]$
  - $\mu = E[X_i]$
- Observed average:
  - $\hat{\mu} = \frac{1}{m}\sum_{i=1}^{m} X_i$

- Chernoff/Hoeffding
  - Additive bound

$$\Pr[|\mu - \hat{\mu}| \geq \epsilon] \leq 2e^{-2m\epsilon^2}$$

  - Relative bounds:

$$\Pr[\hat{\mu} \leq (1-\varepsilon)\mu\ ] \leq e^{-\frac{m\epsilon^2}{2}}$$

$$\Pr[\hat{\mu} \geq (1+\varepsilon)\mu] \leq e^{-\frac{m\epsilon^2}{3}}$$

    - $\varepsilon \in (0,1)$

# Estimation: Sample size

- Let $X_i$ be i.i.d. r.v.
  - $X_i \in [0, R_{max}]$
  - $\mu = E[X_i]$
- Observed average:
  - $\hat{\mu} = \frac{1}{m}\sum_{i=1}^{m} X_i$

- With probability $1 - \delta$
  - We have
    $$|\mu - \hat{\mu}| \leq \epsilon$$
  - For
    $$m \geq \frac{R_{max}^2 \log\frac{2}{\delta}}{2\epsilon^2}$$
- Alternatively,
  - $\epsilon \leq R_{max}\sqrt{\frac{\log 2/\delta}{2m}}$

# Estimating the rewards

- Set $m = \frac{R_{max}^2 \log \frac{2|S|\ |A|}{\delta}}{2\epsilon^2}$
- For each state action $(s, a)$
  - use samples:
  - $R_1(s,a), \dots, R_m(s,a)$
- Estimate using average:

  $\hat{r}(s,a) = \frac{1}{m}\sum_{i=1}^{m} R_i(s,a)$

- For each $(s, a)$:

  With probability $1 - \frac{\delta}{|S|\ |A|}$

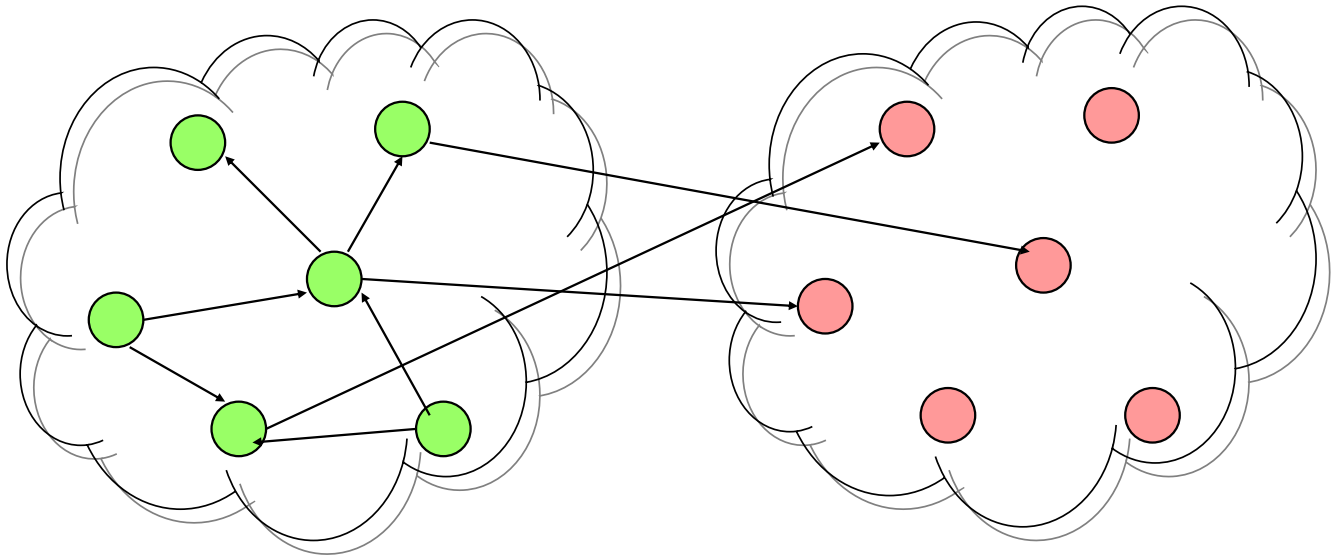
  $|\hat{r}(s,a) - r(s,a)| \leq \epsilon$

- Globally:
  - with probability $1 - \delta$
  - For every $(s, a)$
  - $|\hat{r}(s,a) - r(s,a)| \leq \epsilon$

# Learning Model-Based: on policy

- The learner has control over the action.
  - The immediate goal is to learn a model
- As before:
  - Build an “observed model”:
    - Transition probabilities and Rewards
  - Use the “observed model” to estimate value of the policy.
- Accelerating the learning:
  - How to reach “unexplored” states ?!

# Learning Model-Based: on policy

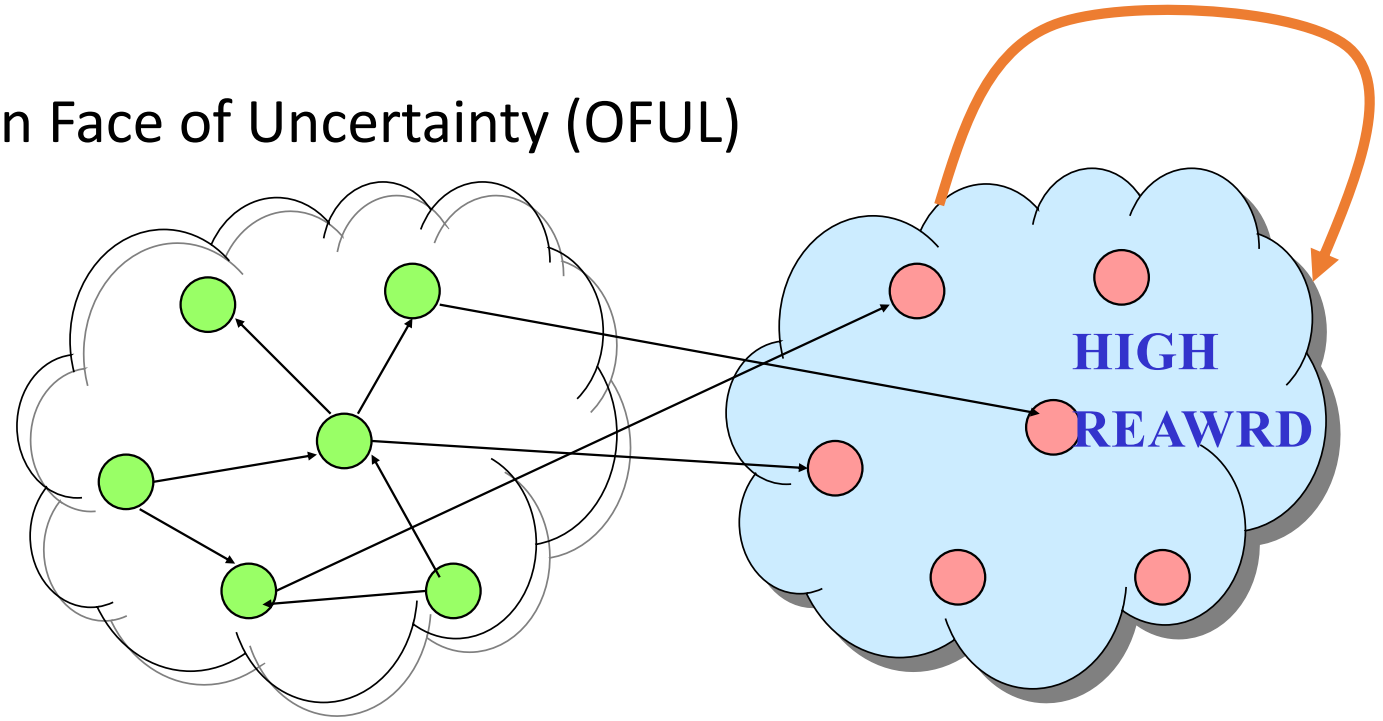


Well sampled states

Relatively unknown states

# Learning - Model Based: on policy

Optimism in Face of Uncertainty (OFUL)




Well sampled states

Relatively unknown states

Exploration → Planning in new MDP

# Learning: Policy improvement

- Assume that we can perform:
  - Given a policy $\pi$,
  - Compute $V^{\pi}$ and $Q^{\pi}$ functions of $\pi$
- Can run policy improvement:
  - $\pi$ = Greedy ($Q^{\pi}$)
- Process converges if estimates are accurate.

# Plan for today

1. Model based RL

2. Q-Learning, SARSA, Monte-Carlo

3. TD Learning for policy evaluation

As this is RL 1 everything is in finite spaces.

# Part 2: outline

- Q-learning:
  - Off-policy
  - Aprrox average online
  - DDP
  - MDP
- SARSA
  - On-policy
- Monte-Carlo
  - Methodology
  - First vs Every visit
  - Control
- Chapter 11 in the book

# Online approx. of mean

- Batch updates:
  - Compute the average
  - $\hat{\mu}_T = \frac{1}{T}\sum_{t=1}^{T} r_t$
  - offline
- Online view
  - $\hat{\mu}_T = \frac{T-1}{T}\hat{\mu}_{T-1} + \frac{1}{T} r_T$
  - $= \hat{\mu}_{T-1} + \frac{1}{T}(r_T - \hat{\mu}_{T-1})$

- Exponential average
  - $\hat{\mu}_T = \hat{\mu}_{T-1} + \alpha_T(r_T - \hat{\mu}_{T-1})$
  - $= \sum_{t=1}^{T} \beta_t r_t$
    - Where:
    - $\beta_t = \alpha_t \prod_{i=t+1}^{T}(1-\alpha_i)$

# Q-learning: motivation

- Learn the optimal Q-function
  - $Q^*(s,a) = r(s,a) + \gamma E_{s' \sim p(\cdot|s,a)}[\max_{a'} Q^*(s',a')]$
    - Discounted return
  - $Q^*$ defines the optimal policy
- Off-policy
  - Observes some exploring policy
    - Need that each $(s,a)$ performed infinitely often
  - Convergence: under mild conditions

# Q-learning: DDP Algorithm

- Initialization: arbitrary
  - $Q_0(s, a) = 0$
- Observe $(s_t, a_t, r_t, s_{t+1})$
- Update:
  - $Q_{t+1}(s_t, a_t) = r_t + \gamma \max_a \{Q_t(s_{t+1}, a)\}$

# Q-learning: DDP

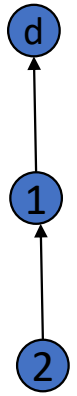
- Theorem:
  - Assume every $(s, a)$ occurs infinitely often
  - Then: $\lim_{t \to \infty} Q_t(s, a) = Q^*(s, a)$
- Proof: Let
  - $\Delta_t = \|Q_t - Q^*\|_\infty = \max_{(s,a)} |Q_t(s, a) - Q^*(s, a)|$

- At time $t$:
  - $|Q_{t+1}(s_t, a_t) - Q^*(s_t, a_t)|$
  - $= |r_t + \gamma \max_{\bar{a}} Q_t(s_{t+1}, \bar{a}) - r_t - \gamma \max_{a'} Q^*(s_{t+1}, a')|$
  - $= \gamma |\max_{\bar{a}} Q_t(s_{t+1}, \bar{a}) - \max_{a'} Q^*(s_{t+1}, a')|$
    - $\max_a\{f(a)\} - \max_b\{g(b)\} \leq \max_a\{f(a) - g(a)\}$
  - $\leq \gamma \max_a |Q_t(s_{t+1}, a) - Q^*(s_{t+1}, a)|$
  - $\leq \gamma \Delta_t$
- Assume that during $[t, t_1]$each $(s, a)$ appear at least once, then,
  - $\Delta_{t_1} \leq \gamma \Delta_t$
- $\Delta_t \to 0$ Q.E.D.

# Q-learning: Shortest paths

- Shortest paths:
  - Assume a single destination
  - The value (cost) is the distance to destination
  - Initialization at very high values
    - Except for destination
- How will convergence occur?
  - By the distance from the destination
  - At any time the Q-cost upper bound real cost.

- Consider the shortest path tree
  - Exists time $t_1$ when nearest node to destination use the edge to destination
    - From that time its best action is fixed!
  - Exists time $t_2 \geq t_1$ when the second nearest node test the edge on the shortest path
    - From that time its best action is fixed!
  - 
  - 
  - Exists time $t_n$ where all nodes …



# Q-learning: MDP Algorithm

- Initialization: arbitrary
  - $Q_0(s, a) = 0$
- Observe $(s_t, a_t, r_t, s_{t+1})$
- Update:
  - $Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha_t(s_t, a_t)\Gamma_t$
  - $\Gamma_t = r_t + \gamma \max_a \{Q_t(s_{t+1}, a)\} - Q_t(s_t, a_t)$

# Q-learning: MDP

- Intuition:
  - If $Q_t = Q^*$ then $E[\Gamma_t] = 0$
- Challenge
  - Using $Q_t$ rather than $Q^*$
    - Not clear that it will converge.
    - The boostrap
  - Need to handle stochastic noise
    - Similar to stochastic gradient descent.

# Q-learning: Convergence

- Theorem: Convergence in the limit
  - If every *(s,a)* performed infinitely often,
  - Step size, for every *(s,a):*

$$\sum_t \alpha_t(s,a) = \infty \quad \text{and} \quad \sum_t \alpha_t^2(s,a) = O(1)$$

**$Q_t(s,a)$ Converges with probability 1 to $Q^*(s,a)$**

# Stochastic Approximation

- Iterative Algorithm:
  - $X_{t+1}(s) = \big(1 - \alpha_t(s)\big)X_t(s) + \alpha_t(s)\big((HX_t)(s) + w_t(s)\big)$
- Well behaved $(B, \gamma)$:
  - Step size: $\sum_{t=1}^{\infty} \alpha(s) = \infty$ and $\sum_{t=1}^{\infty} \alpha^2(s) < \infty$
  - Noise: $E[w_t(s)] = 0$ and $|w_t(s)| \leq B$
  - Contraction: $\exists X^*$: $\|HX - X^*\| \leq \gamma \|X - X^*\|$
    - $HX^* = X^*$

# Stochastic Approximation

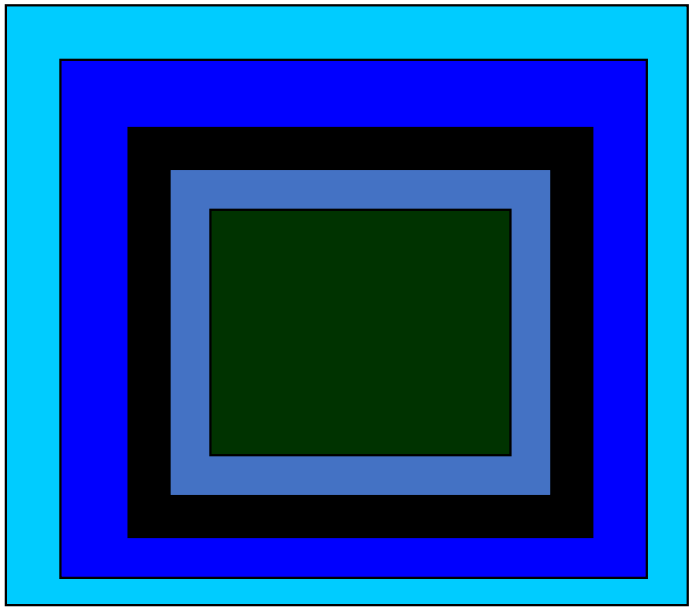
- Theorem:
  - If $X_t$ generated by a well behaved $(B, \gamma)$
  - Then $X_t \rightarrow X^*$
    - With probability 1

# Stochastic Approximation: proof methodology

- Partition the error to two parts:
  - One which contracts, $HX_t$
  - One which is noise, $w_t$
    - the difference of the expected and sampled values.
- The first drops deterministically (as expected).
- The second has zero expectation,
  - bound it using law of large numbers.
    - needs to be bounded for an **entire** next phase.

# SA: Classical Proof Methodology

Let $\mathrm{D}_t = || Q_t - Q^* ||_\infty$



$$D_0 = V_{\max}$$

$$D_1 = (1-\beta)D_0$$

$$D_2 = (1-\beta)D_1$$

$$D_3 = (1-\beta)D_2$$

$$D_4 = (1-\beta)D_3$$

$$D_{t+1} \leq \gamma D_t + 0.5(1-\gamma)D_t = 0.5(1+\gamma)D_t = (1-\beta)D_t$$

# Q-learning: MDP Algorithm

- Initialization: arbitrary
  - $Q_0(s, a) = 0$
- Observe $(s_t, a_t, r_t, s_{t+1})$
- Update:
  - $Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha_t(s_t, a_t)\Gamma_t$
  - $\Gamma_t = r_t + \gamma \max_a \{Q_t(s_{t+1}, a)\} - Q_t(s_t, a_t)$

# Q-learning: contraction

- Operator:
  - $(Hq)(s,a) = \sum_{s'} p(s'|s,a)[r(s,a) + \gamma \max_{b \in A} q(s',b)]$
- Contraction:
  - $\|Hq_1 - Hq_2\|_\infty$
  - $= \gamma \max_{s,a} |\sum_{s'} p(s'|s,a)[\max_{b_1 \in A} q_1(s',b_1) - \max_{b_2 \in A} q_2(s',b_2)]|$
  - $\leq \gamma \max_{s,a} \max_{b,s'} |q_1(s',b) - q_2(s',b)|$
  - $\leq \gamma \|q_1 - q_2\|_\infty$

# Q-learning: convergence

- Rewrite Q-learning:
  - $Q_{t+1}(s_t, a_t) = (1 - \alpha_t(s_t, a_t))Q_t(s_t, a_t) + \alpha_t(s_t, a_t)\Phi_t$
  - $\Phi_t = r_t + \gamma \max_a \{Q_t(s_{t+1}, a)\}$
  - $E[\Phi_t] = (HQ_t)(s_t, a_t)$
  - $w_t(s_t, a_t) = \Phi_t - (HQ_t)(s_t, a_t)$
  - $E[w_t] = 0$, and $|w_t| \leq \frac{R_{max}}{1-\gamma}$
  - $\Phi_t = (HQ_t)(s_t, a_t) + w_t$

# Q-learning: Convergence

- Use *stochastic approximation* convergence
  - Step size: by assumption
  - Noise: $E[w_t] = 0$ & $|w_t| \leq V_{max} = \frac{R_{max}}{1-\gamma}$
  - Contraction:
    - Operator $H$ is $\gamma$-contracting with fix-point $Q^*$
- Result: $Q_t$ converges to $Q^*$ with prob. 1
  - Assumes “exploration”

# Q-learning: Step size

- Step size: $\alpha(s,a) = \frac{1}{g(\#(s,a))}$
- Linear: $g(n) = n$
  - $\sum_{n=1}^{N} \frac{1}{n} \approx \ln N$ and $\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}$
- Poly: $g(n) = n^{\theta}$ for $\theta \in (\frac{1}{2}, 1)$
  - $\sum_{n=1}^{N} \frac{1}{n^{\theta}} \approx \frac{N^{1-\theta}}{1-\theta}$ and $\sum_{n=1}^{\infty} \frac{1}{n^{2\theta}} \leq \frac{1}{2\theta-1} + 1$

# Q-learning: Linear step size

- Bad example for linear step size
- Single state $s$ and action $a$ with $r(s,a) = 0$
- Q-learning initialized $Q_0(s,a) = 1$
  - $Q_t = \left(1 - \frac{1}{t}\right)Q_{t-1} + \frac{1}{t}(0 + \gamma Q_{t-1}) = (1 - \frac{1-\gamma}{t})Q_{t-1}$
  - $Q_t = \Theta(t^{\gamma-1})$
  - For $t = c\left(\frac{1}{\epsilon}\right)^{\frac{1}{1-\gamma}}$ we still have $Q_t \geq \epsilon$

# Q-learning: Poly step size

- Single state $s$ and action $a$ with $r(s,a) = 0$
- Q-learning initialized $Q_0(s,a) = 1$
  - $Q_t = \left(1 - \frac{1}{t^\theta}\right) Q_{t-1} + \frac{1}{t^\theta}(0 + \gamma Q_{t-1}) = (1 - \frac{1-\gamma}{t^\theta}) Q_{t-1}$
  - $Q_t = \Theta\left(e^{-(1-\gamma)t^{1-\theta}}\right)$
  - For $t = c\frac{1}{1-\gamma}\log^{1/(1-\theta)}\frac{1}{\epsilon}$ we have $Q_t \leq \epsilon$

# SARSA: on-policy

- Run Q-learning on-policy
  - Need to select actions
- Policy Requirement:
  - Need to explore
    - Each $(s, a)$ visited infinitely often
  - Need to be Greedy
    - In the limit

# SARSA: Algorithm

- Initialization: arbitrary
  - $Q_0(s, a) = 0$
- Observe $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$
  - $a_{t+1} = \pi(s_{t+1}; Q_t)$ output of the on-policy
- Update:
  - $Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha_t(s_t, a_t)\Gamma_t$
  - $\Gamma_t = r_t + \gamma Q_t(s_{t+1}, a_{t+1}) - Q_t(s_t, a_t)$

# SARSA: exploration

- How to select the action $\pi(s; Q_t)$
  - Let $\bar{a} \in \arg\max_a Q_t(s, a)$
- $\epsilon_t$-greedy:
  - With prob $1 - \epsilon_t$ we have $\pi(s; Q_t) = \bar{a}$
  - $\forall a \in A$: with prob $\frac{\epsilon_t}{|A|}$ we have $\pi(s; Q_t) = a$
  - Value for $\epsilon_t$: $\epsilon_t = \frac{1}{t}$; $\epsilon_t = \frac{1}{t^\theta}$

# SARSA: exploration

- How to select the action $\pi(s; Q_t)$
  - Let $\bar{a} \in \arg\max_a Q_t(s, a)$
- Soft-max:
  - $\forall a \in A$: we have $\pi(s; Q_t) = a$ with prob $\frac{e^{\beta_t Q_t(s,a)}}{\sum_{a' \in A} e^{\beta_t Q_t(s,a')}}$
  - Values for $\beta_t \to \infty$ for $t \to \infty$
    - But slowly

# SARSA: convergence

- Convergence of $Q_t$
  - Similar to $Q$-learning assuming exploration
    - Need enough exploration
- Convergence of return
  - Need to converge to greedy
    - While keeping exploration
  - This is different from Q-learning !

# SARSA: outline convergence of return

- $Q_t \rightarrow Q^{*,\epsilon}$
  - Contracting operator $T^{*,\epsilon}$
- $Q^{*,\epsilon} \approx Q^*$
  - $\|Q^{*,\epsilon} - Q^*\| \leq \frac{\epsilon\gamma V_{max}}{1-\gamma}$
- $\pi_\epsilon$ greedy w.r.t. $Q^{*,\epsilon}$
  - $V^{\pi_\epsilon} \approx V^*$
  - $V^*(s) - V^{\pi_\epsilon}(s) \leq \frac{2\|Q^* - Q^{*,\epsilon}\|}{1-\gamma}$
- $\pi'_\epsilon$ $\epsilon_1$-greedy w.r.t. $Q^{*,\epsilon}$
  - $V^{\pi'_\epsilon} \approx V^{\pi_\epsilon}$
  - $\left|V^{\pi_\epsilon}(s_0) - V^{\pi'_\epsilon}(s_0)\right| \leq \frac{\epsilon_1 R_{max}}{(1-\gamma)^2}$
- Outcome:
  - From some time
  - We play a near optimal policies

# SARSA: convergence of $Q_t$

- Define an operator for $\epsilon$-greedy
  - $(T^{*,\epsilon}q)(s,a) = r(s,a) + \gamma E_{s'}[\frac{\epsilon}{|A|}\sum_{b'} q(s',b') + (1-\epsilon)\max_{a'} q(s',a')]$
- The operator is $\gamma$-contracting
  - $(T^{*,\epsilon}q_1 - T^{*,\epsilon}q_2)(s,a) =$
  - $(1-\epsilon)\gamma E_{s'}[\max_{b'} q_1(s',b') - \max_{b''} q_1(s',b'')]$
  - $+\gamma\frac{\epsilon}{|A|}\sum_{b'} E_{s'}[q_1(s',b') - q_2(s',b')]$
- $\|T^{*,\epsilon}q_1 - T^{*,\epsilon}q_2\| \leq \gamma\|q_1 - q_2\|$

# SARSA: convergence of $Q_t$

- Operator $T^{*,\epsilon}$ converges to a fix point $Q^{*,\epsilon}$
  - Different than $Q^*$, but close to it

❑ $Q^{*,\epsilon}(s,a) = r(s,a) + \gamma E_{s'}\left(\frac{\epsilon}{|A|}\sum_{b'} Q^{*,\epsilon}(s',b') + (1-\epsilon)\max_{a'} Q^{*,\epsilon}(s',a')\right)$

❑ $Q^*(s,a) = r(s,a) + \gamma E_{s'}\left(\max_{a'} Q^*(s',a')\right)$

# SARSA: convergence of $Q_t$

- Let $\Delta^{\epsilon} = \|Q^{*,\epsilon} - Q^{*}\|$
- $Q^{*,\epsilon}(s,a) - Q^{*}(s,a) = (1-\epsilon)\gamma E_{s'}[\max_{a'} Q^{*,\epsilon}(s',a') - \max_{a''} Q^{*}(s',a'')]$
  - $+\gamma \frac{\epsilon}{|A|} \sum_{b'} E_{s'}[Q^{*,\epsilon}(s',b') - Q^{*}(s',b')]$
- $\leq \gamma(1-\epsilon)\Delta^{\epsilon} + \epsilon\gamma V_{max}$
- Therefore: $\Delta^{\epsilon} \leq (\epsilon\gamma V_{max})/(1-\gamma)$
- Theorem: Algorithm SARSA with $\epsilon$-greedy converges to $Q^{*,\epsilon}$, where $\|Q^{*,\epsilon} - Q^{*}\| \leq (\epsilon\gamma V_{max})/(1-\gamma)$

# Convergence of the return

- Lemma:
  - Q: $|| Q - Q^* ||_\infty \leq \Delta$
  - $\pi$ greedy policy w.r.t. Q: $\pi(s) \in \underset{a}{\text{argmax}}\, Q(s,a)$
- Then,
  - $V^*(s) - V^\pi(s) \leq \frac{2\Delta}{1-\gamma}$

- Proof:
  - $V^*(s) - Q^*\big(s, \pi(s)\big) = Q^*(s, a^*) - Q^*\big(s, \pi(s)\big) \leq 2\Delta$:
    - $\left|Q^*\big(s, \pi(s)\big) - Q\big(s, \pi(s)\big)\right| \leq \Delta$
    - $|Q^*(s, a^*) - Q(s, a^*)| \leq \Delta$ , $a^*$ optimal action
    - $Q\big(s, \pi(s)\big) \geq Q(s, a^*)$, $\pi$ is greedy w.r.t. Q
  - $V^*(s) = Q^*(s, a^*) \leq Q^*(s, \pi(s)) + 2\Delta = E[r_0] + \gamma E[V^*(s_1)] + 2\Delta$
    - $r_0 = R(s, \pi(s))$, $s_1$ next state using $\pi(s)$
  - General: $V^*(s) \leq E\left[\sum_i \gamma^i r_i\right] + \sum_i 2\Delta\gamma^i = V^\pi(s) + \frac{2\Delta}{1-\gamma}$
- QED

- Lemma: Let $\pi$ and $\pi'_\epsilon$, such that for any state $s$

$\left|\left|\pi(\cdot\,|s) - \pi'_\epsilon(\cdot\,|s)\right|\right|_1 \leq \epsilon_1$. Then,

$$\left|V^{\pi}(s_0) - V^{\pi'_\epsilon}(s_0)\right| \leq \frac{\epsilon_1 \gamma R_{max}}{(1-\gamma)^2}$$

- Proof: $\pi_\epsilon$ agrees with $\pi$ at time t with prob at least $(1-\epsilon_1)^t$
  - $V^{\pi}(s_0) - V^{\pi'_\epsilon}(s_0) \leq E[\sum_t \gamma_t r_t] - E[\sum_t (1-\epsilon_1)^t \gamma^t r_t]$
  - $\leq \sum_t \gamma^t R_{max} - \sum_t (1-\epsilon_1)^t \gamma^t R_{max}$
  - $\frac{R_{max}}{1-\gamma} - \frac{R_{max}}{1-\gamma(1-\epsilon_1)} \leq \frac{\epsilon_1 \gamma R_{max}}{(1-\gamma)^2}$

# Monte-Carlo

# Monte Carlo: motivation

- Directly learn from experience
- Model free
- Simplest idea:
  - Average the returns
  - Learn a value function
  - No dependencies
    - Both plus and minus
- Mainly for episodic MDP

# Monte Carlo: basic idea

- Learn $V^{\pi}$ from episodes generated by $\pi$
  - Episode: $s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_k, a_k, r_k, s_{k+1}$
    - $a_t \sim \pi(\cdot \mid s_t)$
  - Return (of an episode):
    - $V^{\pi}(s) = E^{\pi}[\sum_{t=1}^{k} r_t \mid \mathrm{s}_1 = \mathrm{s}]$
    - Observed rewards: $G(s) = \sum_{t=1}^{k} r_t$

# Monte Carlo: Algorithm

- Given observations
  - $G_1(s), \ldots, G_m(s)$
  - Estimate $\hat{V}^{\pi}(s) = \frac{1}{m} \sum_{i=1}^{m} G_i\,(s)$
- How do we generate the samples:
  - How to define $G_i(s)$ from episodes

# MC: Generating samples

- Initial state:
  - Use only the initial state of the trajectory
  - Given Episode: $s_1, a_1, r_1, s_2, a_2, r_2, \ldots, s_k, a_k, r_k$
  - Update only $\hat{V}^{\pi}(s_1)$
- Weaknesses?
  - Not clear that we will update all states
  - Very wasteful
    - Can we do better?

# MC: First visit

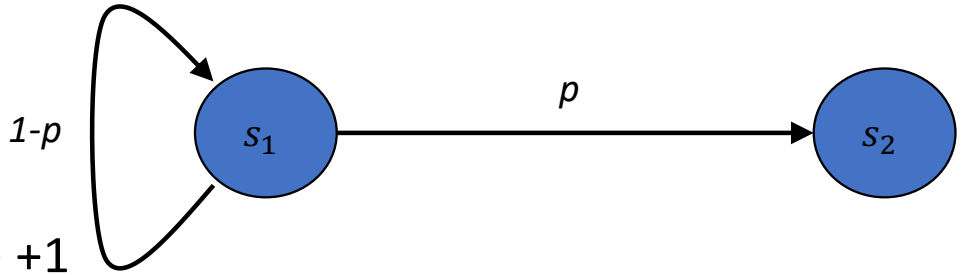
- Given the $i^{th}$ episode:
  - $s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_k, a_k, r_k$
  - For each state $s_j$ that appears in the episode
  - Update $\hat{V}^{\pi}(s_j)$ once
    - Use only the first occurrence of $s_j$
      - Assume that this is the $m$-th state
    - $G_i(s_j) = \sum_{l=m}^{k} r_l$
  - Unbiased estimator

# MC: Every visit

- Given the $i^{th}$ episode:
  - $s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_k, a_k, r_k$
  - update **each** appearance of state $s_j$
  - Update $\hat{V}^{\pi}(s_j)$ using multiple suffixes
    - For each occurrence as the $m$-th state
    - $G_i(s_j) = \sum_{l=m}^{k} r_l$
- A state updated multiple times in an episode
  - Updates are correlated!

# First versus Every visit

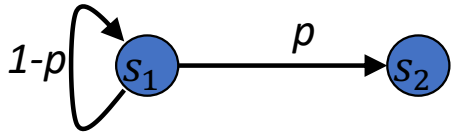
Test case:




- Rewards are +1
- $V(s_1) = \frac{1}{p}$

# First versus Every visit (Test case, cont')

- We observe a trajectory:
  - $s_1, s_1, s_1, s_1, s_2$
- What would be your estimate for $V(s_1)$
  - What about $p$?
- First visit: $\hat{V}(s_1) = 4$
- Every visit: $\hat{V}(s_1) = \frac{4+3+2+1}{4} = 2.5$



# Maximum Likelihood model (Test case, cont')

- Maximum Likelihood model:
  - $p^* = \arg\max(1-p)^3 p$
    - $(1-p^*)^3 - 3(1-p^*)^2 p^* = 0$
  - $p^* = \frac{1}{4}$
- Expected value:
  - $V(s_1) = \frac{1}{p^*} = 4$
  - Coincides here with first visit
    - but not always!

# Max Likelihood and First visit (Test case, cont')

- Consider a state $s$
- First visit when updating $s$ ignores:
  - Trajectories that not include $s$
  - All states up to first occurance of $s$
- Reduced sample: ignore those parts
- Theorem:

First Visit identical to MaxLikelihood on reduced sample.

- Proof:
- Let $G_i$ return of episode $i \in [N]$
- First visit:
  - $\frac{1}{N}\sum_j G_j = \frac{1}{N}\sum_j \sum_i r_{i,j}$
- Notation:
  - $n(s)$: number of visits to $s$
  - $n(u,v)$: transitions from $u$ to $v$
- Maximum likelihood model:
  - Identical to the observed model
  - $\hat{p}(u,v) = n(u,v)/n(u)$
  - $\hat{r}(v) = \frac{1}{n(v)}\sum_i r_i^v$

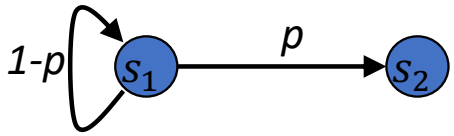
- $\mu(s)$: probability of reaching $s$
  - Need to show that it is $n(s)/N$
- Assume $\mu(s) = n(s)/N$ :
  - $V^\pi = \sum_v \mu(v)\hat{r}(v)$
    $= \sum_v \frac{n(v)}{N}\frac{1}{n(v)}\sum_i r_i^v = \frac{1}{N}\sum_j G_j$
- Computing $\mu(s)$
  - Solves the dynamics with $\hat{p}(u,v)$
  - $\mu(v) = \sum_u \hat{p}(u,v)\ \mu(u)$
  - For $\mu(s) = n(s)/N$
  - $n(v) = \sum_u n(u,v)$

# Every visit minimizes MSE

- Let: $s_{i,j}$: $j^{th}$ state in the $i^{th}$ trajectory. $R(s_{i,j}) = \sum_{t \geq j} r(s_{i,t})$
  - SE: $\sum_{s_{i,j}} \left(\hat{V}(s_{i,j}) - R(s_{i,j})\right)^2$
  - $\sum_s \sum_{i,j:s_{i,j}=s} \left(\hat{V}(s) - R(s_{i,j})\right)^2$
  - The minimizer $V'(s) = \frac{\sum_{i,j:s_{i,j}=s} R(s_{i,j})}{|\{(i,j):s_{i,j}=s\}|}$
    - Exactly MC Every visit.

# First versus Every visit

- First visit: Unbiased
  - episode $n$ times $s_1$
  - $FV = n$
  - $E[FV] = \frac{1}{p}$




- Every visit: Biased
  - episode $n$ times $s_1$
  - $EV = \frac{n(n+1)}{2}\frac{1}{n} = \frac{n+1}{2}$
  - $E[EV] = \frac{1}{2p} + \frac{1}{2}$
- Multiple episodes
  - $EV = \frac{sum\ episodes\ s_1}{\#(s_1)}$
  - $E[EV] = \frac{E[\frac{n(n+1)}{2}]}{E[n]} = \frac{1}{p}$

# Monte Carlo: control

- Learn Q-function
- For every $(s, a)$:
  - Update $\hat{Q}(s, a)$ using episodes
    - Either first or every visit
- Control:
  - Update policy $\pi$ to be near-greedy
    - For example, $\epsilon$-greedy

# Policy improvement: Epsilon-greedy

- Policy improvement:
  - $\pi'(\cdot\,|s)$ sets $\bar{a} = \arg\max_a Q^{\pi}(s,a)$
- $\epsilon$-greedy improvement policy

$$\pi''(a|s) = \begin{cases} \frac{\epsilon}{|A|} + (1-\epsilon) & for\ a = \bar{a} \\ \frac{\epsilon}{|A|} & otherwise \end{cases}$$

# Policy improvement: Epsilon-greedy

- Theorem: For any $\epsilon$-greedy $\pi$ the $\epsilon$-greedy improvement policy $\pi'$ has $V^{\pi'} \geq V^{\pi}$
- Proof:
  - $E_{a\sim\pi'}[Q^{\pi}(s,a)] = \sum_{a\in A} \pi'(a|s) Q^{\pi}(s,a)$
  - $= \frac{\epsilon}{A} \sum_{a\in A} Q^{\pi}(s,a) + (1-\epsilon) Q^{\pi}(s,\bar{a})$
  - $\geq \frac{\epsilon}{A} \sum_{a\in A} Q^{\pi}(s,a) + (1-\epsilon) \sum_{a} \frac{\pi(a|s) - \epsilon/|A|}{1-\epsilon} Q^{\pi}(s,a)$
  - $= \sum_{a\in A} \pi(a|s) Q^{\pi}(s,a) = V^{\pi}(s)$
  - Similar to basic policy improvement we show:
    - $V^{\pi'}(s) \geq E_{a\sim\pi'}[Q^{\pi}(s,a)] \geq V^{\pi}(s)$

# Monte Carlo: pros and cons

- Pros:
  - Does not assume Markovian environment
  - Extends naturally for function approximation
  - Unbiased (first visit)
- Cons:
  - Update only at the end of a complete episode
  - Biased (every visit)

# Plan for today

1. Model based RL

2. Q-Learning, SARSA, Monte-Carlo

3. TD Learning for policy evaluation

As this is RL#1 everything is in finite spaces.

# Part 3: outline

- Temporal Differences:
  - TD(0)
  - TD($\lambda$)
  - $SARSA(\lambda)$
- Importance Sampling
  - Behavioral policy
  - Evaluated policy
- Actor-Critic

# On-policy versus Off-policy

**On-policy**

- Algorithm selects actions
- One algorithm for
  - Actions
  - Updates

**Off-policy**

- Separation between selecting
  - Actions
    - Behavioral policy
    - Logs
  - Updates
    - Current belief

# Q-learning: Off-Policy Algorithm

- Initialization: arbitrary
  - $Q_0(s, a) = 0$
- Observe $(s_t, a_t, r_t, s_{t+1})$
- Update:
  - $Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha_t(s_t, a_t)\Gamma_t$
  - $\Gamma_t = r_t + \gamma \max_a \{Q_t(s_{t+1}, a)\} - Q_t(s_t, a_t)$

# SARSA: On-Policy Algorithm

- Initialization: arbitrary
  - $Q_0(s, a) = 0$
- Observe $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$
  - $a_{t+1} = \pi(s_{t+1}; Q_t)$ output of the on-policy
- Update:
  - $Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha_t(s_t, a_t)\Gamma_t$
  - $\Gamma_t = r_t + \gamma Q_t(s_{t+1}, a_{t+1}) - Q_t(s_t, a_t)$

# Temporal Differences

# TD: methods

- Learns value function
  - directly from experience
- Model-free
- Uses incomplete episodes
  - Updates before the end
- Updates the estimate
  - Given the change in estimates

# TD(0): inspiration

- Fix a policy $\pi$
- Value Iteration
  - $V_{t+1}(s) = E^{\pi}[r(s,a) + \gamma V_t(s')]$
    - Convergence $V_t \to V^{\pi}$
- Assume we sample $(s_t, a_t, r_t, s_{t+1})$
  - $E^{\pi}\left[r_t + \gamma \hat{V}_t(s_{t+1}) \middle| s_t\right] = E^{\pi}[r(s,a) + \gamma \hat{V}_t(s')]$
    - Where $s = s_t$, $a = a_t = \pi(s_t)$, and $s' = s_{t+1}$
  - TD(0): Do an update in that direction

# TD(0): Algorithm

- Maintains $\hat{V}$
- Update using $(s_t, a_t, r_t, s_{t+1})$
  - $\hat{V}(s_t) = (1 - \alpha_t)\hat{V}(s_t) + \alpha_t[r_t + \gamma\hat{V}(s_{t+1})]$
  - $\hat{V}(s_t) = \hat{V}(s_t) + \alpha_t[r_t + \gamma\hat{V}(s_{t+1}) - \hat{V}(s_t)]$
    - Step size $\alpha_t(s, a)$
- TD(0): $\hat{V}(s_t) = \hat{V}(s_t) + \alpha_t\Delta_t$
  - $\Delta_t = r_t + \gamma\hat{V}(s_{t+1}) - \hat{V}(s_t)$

# Empirical model

- Empirical model
  - Given $(s_t, a_t, r_t, s_{t+1})$
    - $\hat{r}(s,a) = \frac{1}{n(s,a)} \sum_{s_t=s, a_t=a} r_t$
      - $n(s,a) = |\{t: s_t = s, a_t = a\}|$
    - $\hat{p}(s'|s,a) = \frac{n(s,a,s')}{n(s,a)}$
      - $n(s,a,s') = |\{t: s_t = s, a_t = a, s_{t+1} = s'\}|$

- Recall: Empirical Model = Maximum Likelihood Model

# TD(0) and empirical model

- Theorem:
  - Run TD(0) on the sample until convergence
    - Re-sampling from buffer
  - Then TD(0) estimate $\hat{V}^{\pi}$ equal to $V^{\pi}$ in the empirical model

# TD(0): Convergence

- Theorem:
  - Step size, for every *(s,a):*

$$\sum_t \alpha_t(s,a) = \infty \quad \text{and} \quad \sum_t \alpha_t^2(s,a) = O(1)$$

  - **$V_t(s)$ Converges with probability 1 to $V^{\pi}(s)$**

- Very similar to Q-learning.

# TD(0): contraction

- Operator (for a given policy $\pi$)
  - $(Hv)(s) = r(s,a) + \gamma \sum_{s'} p(s'|s,a)v(s')$
    - Where $a = \pi(s)$
- Contraction:
  - $\|Hv_1 - Hv_2\|_\infty \leq \gamma \max_{s,a} |\sum_{s'} p(s'|s,a)[v_1(s') - v_2(s')]|$
  - $\leq \gamma \max_{s,a} \max_{s'} |v_1(s') - v_2(s')|$
  - $\leq \gamma \|v_1 - v_2\|_\infty$

# TD(0): convergence

- Rewrite TD(0):
  - $V_{t+1}(s_t) = (1-\alpha_t)V_t(s_t) + \alpha_t \Phi_t$
  - $\Phi_t = r_t + \gamma V_t(s_{t+1})$
  - $E[\Phi_t] = (HV_t)(s_t)$
  - $w_t(s_t) = \Phi_t - (HV_t)(s_t)$
    - $E[w_t] = 0$, and $|w_t| \leq \frac{R_{max}}{1-\gamma}$
  - $\Phi_t = (HV_t)(s_t) + w_t(s_t)$

# TD(0): Convergence

- Use *stochastic approximation* convergence
  - Step size: by assumption
  - Noise: $E[w_t] = 0$ & $|w_t| \leq V_{max} = \frac{R_{max}}{1-\gamma}$
  - Contraction:
    - operator $H$ is $\gamma$-contracting with $V^\pi$ fixed-point
- Result: $V_t$ converges to $V^\pi$ with prob. 1

# TD(0) vs MC vs DP

- Three ways to look at the value function $V^{\pi}(s)$
  - (MC): $E^{\pi}[R_t|s_t = s] = E^{\pi}[\sum_{i=t}^{\infty} r_i \,|s_t = s]$
  - TD(0): $E^{\pi}[r(s,a) + \gamma V^{\pi}(s')|s_t = s, a_t = a, s' = s_{t+1}]$
  - $(DP)$: $\sum_{a\in A} \pi(a|s)[r(s,a) + \gamma \sum_{s'} p(s'|s,a)V^{\pi}(s')]$

# Part 3: outline

- Temporal Differences:
  - TD(0)
  - TD($\lambda$)
  - $SARSA(\lambda)$
- Importance Sampling
  - Behavioral policy
  - Evaluated policy
- Actor-Critic

# TD: multiple look-ahead

# TD: multiple look-ahead

- TD(0) update:
  - Given $(s_t, a_t, r_t, s_{t+1})$
  - $\Delta_t = R_t^{(1)}(s_t) - \hat{V}(s_t)$
    - $R_t^{(1)}(s_t) = r_t + \gamma \hat{V}(s_{t+1})$
- Two step look-ahead
  - Given $(s_t, a_t, r_t, s_{t+1}, a_{t+1}, r_{t+1}, s_{t+2})$
  - $R_t^{(2)}(s_t) = r_t + \gamma r_{t+1} + \gamma^2 \hat{V}(s_{t+2})$

# TD: multiple look-ahead

- Generalize to *n* steps:
  - $R_t^{(n)}(s_t) = \sum_{i=0}^{n-1} \gamma^i r_{t+i} + \gamma^n \hat{V}(s_{t+n})$
- New updates:
  - $\Delta_t^{(n)} = R_t^{(n)}(s_t) - \hat{V}(s_t)$
  - $\Delta_t^{(n)} = \sum_{i=0}^{n-1} \gamma^i \Delta_{t+i}$
    - $= \sum_{i=0}^{n-1} \gamma^i [r_{t+i} + \gamma \hat{V}(s_{t+i+1}) - \hat{V}(s_{t+i})]$
    - $= \sum_{i=0}^{n-1} \gamma^i r_{t+i} + \sum_{i=0}^{n-1} \gamma^{i+1} \hat{V}(s_{t+i+1}) - \sum_{i=0}^{n-1} \gamma^i \hat{V}(s_{t+i})$
    - $= \sum_{i=0}^{n-1} \gamma^i r_{t+i} + \gamma^n \hat{V}(s_{t+n}) - \hat{V}(s_t) = R_t^{(n)}(s_t) - \hat{V}(s_t) = \Delta_t^{(n)}$

# TD: multiple look-ahead

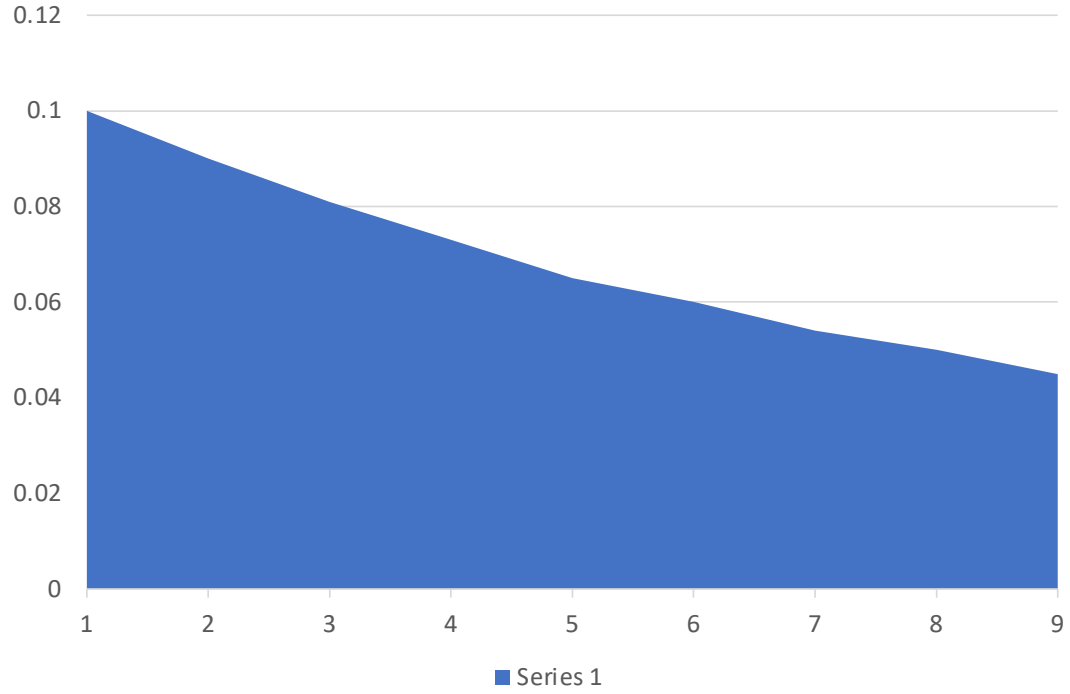
- Using *n* step look-ahead
  - $\hat{V}(s_t) = \hat{V}(s_t) + \alpha_t \Delta_t^{(n)}$
  - $\Delta_t^{(n)} = R_t^{(n)}(s_t) - \hat{V}(s_t)$
- The operator $R_t^{(n)}$ is $\gamma^n$-contracting
  - $\left\| R_t^{(n)}(V_1) - R_t^{(n)}(V_2) \right\| \leq \gamma^n \|V_1 - V_2\|_\infty$

# TD: how to select the look-ahead

- We can use any parameter *n*
  - If episodes ends before, pad with zeros
  - Actually, $n = \infty$ is simply MC
- Basic idea: we can average over *n*
  - What is the best way to average?
  - The simplest is exponential averaging!
    - $(1 - \lambda) \sum_n \lambda^{n-1} \Delta^{(n)}$

# TD: exponential averaging

- Have a parameter $\lambda \in (0,1)$
  - Weight of $n \geq 1$ is $(1-\lambda)\lambda^{n-1}$

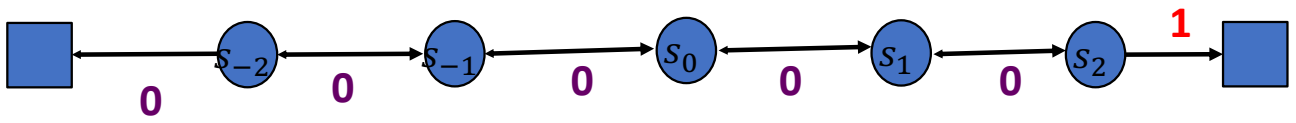


# $TD(\lambda)$

- Recall:
  - $\Delta_t^{(n)} = R_t^{(n)}(s_t) - \hat{V}(s_t)$
- $TD(\lambda)$ update:
  - $\hat{V}(s_t) = \hat{V}(s_t) + \alpha_t(1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}\ \Delta_t^{(n)}$
    - $\hat{V}(s_t) = \hat{V}(s_t) + \alpha_t(1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}\sum_{i=0}^{n-1}\gamma^i\Delta_{t+i}$
    - $\hat{V}(s_t) = \hat{V}(s_t) + \alpha_t\sum_{i=0}^{\infty}\lambda^i\gamma^i\Delta_{t+i}$
- Do not confuse $\gamma$ and $\lambda$

# Random walk example

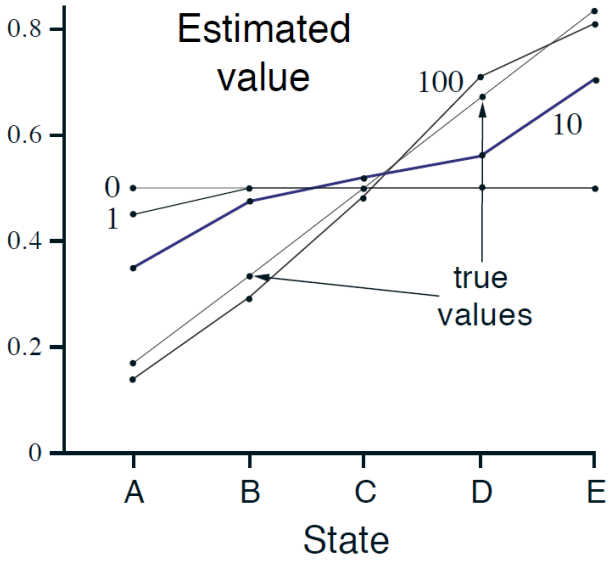
- Random walk on a line

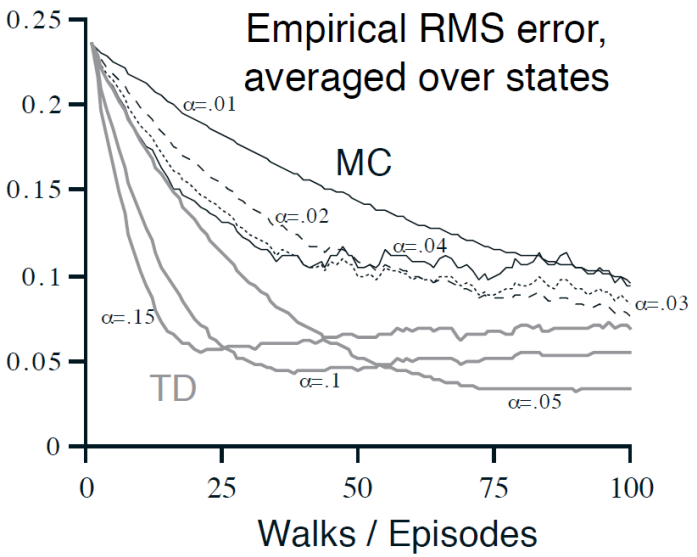



- Policy: random
  - Left/right with prob 0.5
- Values:

$$V(s_{-2}) = \frac{1}{6}; V(s_{-1}) = \frac{2}{6}; V(s_0) = \frac{3}{6}; V(s_1) = \frac{4}{6}; V(s_2) = \frac{5}{6}$$

# Random walk: TD(0) vs MC



# Part 3: outline

- Temporal Differences:
  - TD(0)
  - TD($\lambda$)
  - $SARSA(\lambda)$
- Importance Sampling
  - Behavioral policy
  - Evaluated policy
- Actor-Critic

# $TD(\lambda)$: Forward view

- $TD(\lambda)$ update:
  - $\hat{V}(s_t) = \hat{V}(s_t) + \alpha_t(1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}\ \Delta_t^{(n)}$
    - $\hat{V}(s_t) = \hat{V}(s_t) + \alpha_t \sum_{i=0}^{\infty}\lambda^i \gamma^i \Delta_{t+i}$
  - $\Delta_t^{(n)} = R_t^{(n)}(s_t) - \hat{V}(s_t)$
- Look-ahead:
  - How can we know future rewards
  - Do we need to wait?
  - For $TD(\lambda)$ we depend on all future rewards!

# $TD(\lambda)$: Backward view

- Forward view
  - Theoretical justification
- Backward view:
  - Online updates
    - Every time
    - Using incomplete information
- Performance: They will be equivalent
  - At the end of the episode

# $TD(\lambda)$ backward updates

- Basic idea
  - The updates are linear in Temporal Differences
  - Fix a time $t$ and state $s = s_t$
    - TD: $\Delta_t = r_t + \gamma V_t(s_{t+1}) - V_t(s_t)$
    - How is it weighted by previous time steps?

- Fix state $s$
  - Forward influences only $s_\tau = s$
    - Time $t$ influences by $(\gamma\lambda)^{t-\tau}\Delta_t$
      - Recall $\sum_{n=1}^{\infty} \lambda^{n-1}\ \Delta_\tau^{(n)} = \sum_{i=1}^{\infty} \lambda^i \gamma^i \Delta_{\tau+i}$
  - We can sum all the influences
    - $e_t(s) = \sum_{\tau=0}^{t} (\gamma\lambda)^{t-\tau}\, I(s_\tau = s)$

# $TD(\lambda)$: Eligibility traces

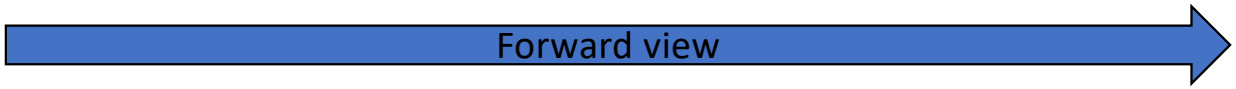
- Define eligibility trace
  - $e_t(s) = \sum_{\tau=0}^{t} (\gamma\lambda)^{t-\tau} I(s_\tau = s)$
- Compute it online:
  - $e_t(s) = \lambda\gamma e_{t-1}(s) + I(s_t = s)$
- Update
  - $\hat{V}_{t+1}(s) = \hat{V}_t(s) + \alpha_t e_t(s)\Delta_t$
- TD(0): $\hat{V}_{t+1}(s) = \hat{V}_t(s) + \alpha_t I(s_t = s)\Delta_t$

# $TD(\lambda)$: algorithm

- Initialization
  - Arbitrary $\hat{V}(s)$, $e_0(s) = 0$
- Update: observe $(s_t, a_t, r_t, s_{t+1})$
  - $\Delta_t = r_t + \gamma \hat{V}_t(s_{t+1}) - \hat{V}_t(s_t)$
  - $e_t(s) = \gamma \lambda e_{t-1}(s) + I(s_t = s)$
  - $\hat{V}_{t+1}(s) = \hat{V}_t(s) + \alpha_t e_t(s) \Delta_t$

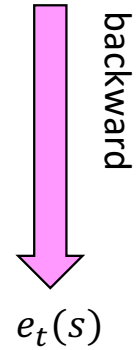
# $TD(\lambda)$: Equivalence of Forward and backward view

- Update for state $s$




$s_0 = s$
$s_2 = s$
$s_4 = s$

| $\Delta_0$ | $\Delta_1$ | $\Delta_2$ | $\Delta_3$ | $\Delta_4$ | $\Delta_5$ | $\Delta_6$ | $\Delta_7$ | $\Delta_8$ | $\Delta_9$ |
|---|---|---|---|---|---|---|---|---|---|
| 1 | $\gamma\lambda$ | $(\gamma\lambda)^2$ | $(\gamma\lambda)^3$ | $(\gamma\lambda)^4$ | $(\gamma\lambda)^5$ | $(\gamma\lambda)^6$ | $(\gamma\lambda)^7$ | $(\gamma\lambda)^8$ | $(\gamma\lambda)^9$ |
| | | 1 | $\gamma\lambda$ | $(\gamma\lambda)^2$ | $(\gamma\lambda)^3$ | $(\gamma\lambda)^4$ | $(\gamma\lambda)^5$ | $(\gamma\lambda)^6$ | $(\gamma\lambda)^7$ |
| | | | | 1 | $\gamma\lambda$ | $(\gamma\lambda)^2$ | $(\gamma\lambda)^3$ | $(\gamma\lambda)^4$ | $(\gamma\lambda)^5$ |
| $e_0(s)$ | $e_1(s)$ | $e_2(s)$ | $e_3(s)$ | $e_4(s)$ | $e_5(s)$ | $e_6(s)$ | $e_7(s)$ | $e_8(s)$ | $e_9(s)$ |



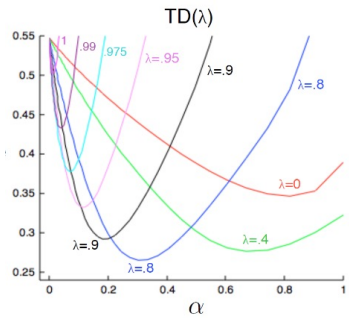
# $TD(\lambda)$: Equivalence of Forward and backward view

- Forward view
  - $\Delta V_t^F(s) = \alpha[R_t^\lambda - V_t(s)]$
    - $R_t^\lambda = (1-\lambda)\sum_{n=1}^{\infty} \lambda^{n-1} R_t^{(n)}$
- Backward view
  - $\Delta V_t^B(s) = \alpha e_t(s)\Delta_t$
    - $e_t(s) = \sum_{k=0}^{t} (\lambda\gamma)^{t-k} I(s = s_k)$
- Theorem: $\sum_{t=0}^{\infty} \Delta V_t^B(s) = \sum_{t=0}^{\infty} \Delta V_t^F(s) I(s_t = s)$

- Theorem:
  - $\sum_{t=0}^{\infty} \Delta V_t^B(s) = \sum_{t=0}^{\infty} \Delta V_t^F(s) I(s_t = s)$
- Proof sketch: Forward view
  - $\sum_{t=0}^{\infty} \Delta V_t^F(s)\, I(s = s_t)$
  - $= \sum_{t=0}^{\infty} \alpha\,(1-\lambda) \sum_{n=t}^{\infty} \lambda^{n-t}\, \Delta_t^{(n)} I(s = s_t)$
  - $= \sum_{n \geq t} \alpha(\gamma\lambda)^{n-t} \Delta_n(s) I(s = s_t)$
- Backward view
  - $\sum_{t=0}^{\infty} \Delta V_t^B(s)$
  - $= \sum_{t=0}^{\infty} \alpha \Delta_t(s) \sum_{n=0}^{t} (\gamma\lambda)^{t-n} I(s = s_n)$
  - $= \sum_{t \geq n} \alpha(\gamma\lambda)^{t-n} \Delta_t(s) I(s = s_n)$
- Interchange: $n \leftrightarrow t$

# $TD(\lambda)$: Summary

- Updates more frequently frequent states
  - Higher eligibility traces $e_t$
- Single parameter $\lambda$
  - Has the potential to improve over both:
    - TD(0)
    - MC
- Convergence:
  - can be shown



# Part 3: outline

- Temporal Differences:
  - TD(0)
  - TD($\lambda$)
  - $SARSA(\lambda)$
- Importance Sampling
  - Behavioral policy
  - Evaluated policy
- Actor-Critic

# SARSA

# $SARSA$: $n$-step look-ahead

- Another application of eligibility traces
- SARSA update
  - $r(s_t, a_t) + \gamma Q_t(s_{t+1}, a_{t+1}) - Q_t(s_t, a_t)$
- SARSA with $n$-step look-ahead
  - $q_t^{(n)} = \sum_{i=0}^{n-1} \gamma^i r(s_{t+i}, a_{t+i}) + \gamma^n Q_t(s_{t+n}, a_{t+n})$
  - $Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha_t(q_t^{(n)} - Q_t(s_t, a_t))$

# $SARSA(\lambda)$: Forward view

- Let $q_t^{\lambda}$ be a weighted average of $q_t^{(n)}$
  - Using exponential weights
  - $q_t^{\lambda} = (1-\lambda)\sum_{n=1}^{\infty} \lambda^{n-1} q_t^{(n)}$
- Forward view of $SARSA(\lambda)$
  - $Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha_t(q_t^{\lambda} - Q_t(s_t, a_t))$

# $SARSA(\lambda)$: Backward view

- Use eligibility traces
  - $e_0(s,a) = 0$
  - $e_t(s,a) = \gamma\lambda e_{t-1}(s,a) + I(s_t = s, a_t = a)$
- Update of $Q(s,a)$
  - $\Delta_t = r_t + \gamma Q_t(s_{t+1}, a_{t+1}) - Q_t(s_t, a_t)$
  - $Q_{t+1}(s,a\ ) = Q_t(s,a) + \alpha_t \Delta_t e_t(s,a)$

# Importance sampling

# Importance sampling

- Using one distribution to evaluate another
  - Sampling using $Q$
  - Evaluation expectation w.r.t. $P$
- Derivation
  - $E_{X\sim P}[f(X)] = \sum_x P(x)f(x)$
  - $= \sum_x Q(x)\frac{P(x)}{Q(x)}f(x)$
  - $= E_{X\sim Q}[\frac{P(X)}{Q(X)}f(X)]$

# Importance sampling

- Unbiased estimator !
  - Expectation is perfect
- Variance
  - Can be huge
    - Depends on $P(x)/Q(x)$

# Importance sampling: MC

- Action selection policy $\pi$
- Evaluated policy $\rho$
- Estimated return
  - $\frac{\rho(s_1,a_1,r_1,\ldots,s_T,a_T,r_T)}{\pi(s_1,a_1,r_1,\ldots,s_T,a_T,r_T)} = \prod_{t=1}^{T} \frac{\rho(a_t|s_t)}{\pi(a_t|s_t)} \prod_{t=1}^{T} \frac{p(s_{t+1}|s_t,a_t)}{p(s_{t+1}|s_t,a_t)} \prod_{t=1}^{T} \frac{R(r_t|s_t,a_t)}{R(r_t|s_t,a_t)}$
  - $G^{\rho/\pi} = \prod_{t=1}^{T} \frac{\rho(a_t|s_t)}{\pi(a_t|s_t)} (\sum_{t=1}^{T} r_t)$
  - $\hat{V}^{\rho}(s_1) \leftarrow \hat{V}^{\rho}(s_1) + \alpha(G^{\rho/\pi} - \hat{V}^{\rho}(s_1))$
- Might have huge updates

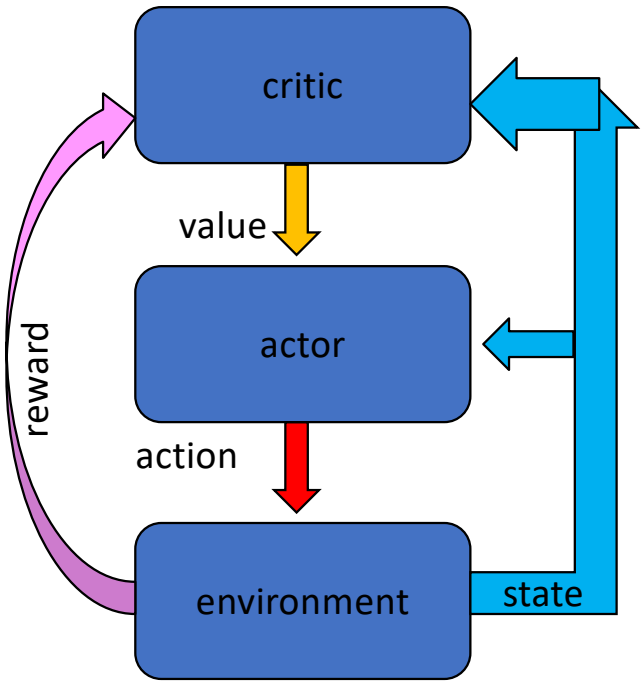
# Importance sampling: TD

- Action selection policy $\pi$
- Evaluated policy $\rho$
- Re-weight the observation
  - $r_t + \gamma \hat{V}^{\rho}(s_{t+1})$
  - $\Delta_t = \frac{\rho(a_t|s_t)}{\pi(a_t|s_t)}\left[r_t + \gamma \hat{V}^{\rho}(s_{t+1})\right] - \hat{V}^{\rho}(\mathrm{s_t})$
- Much smaller variance!

# Actor-Critic

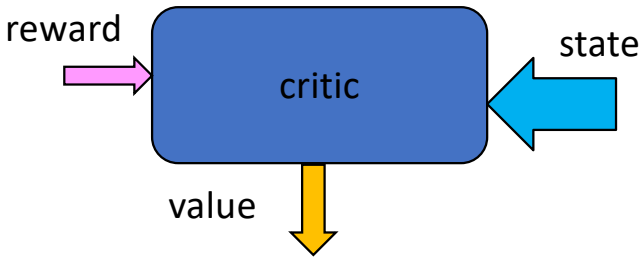
# Actor-Critic Methodology

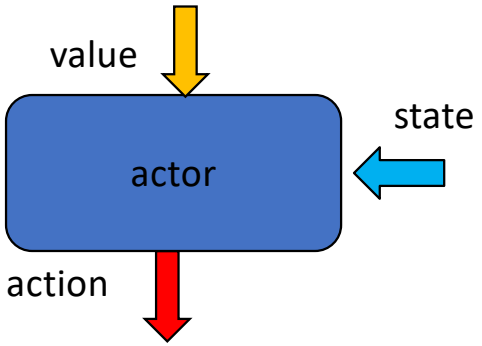
- A general methodology to design RL algorithms:
  - Policy improvements
- Critic:
  - Computes value function
- Actor
  - Selects action
  - Improves policy



# Actor-Critic Methodology

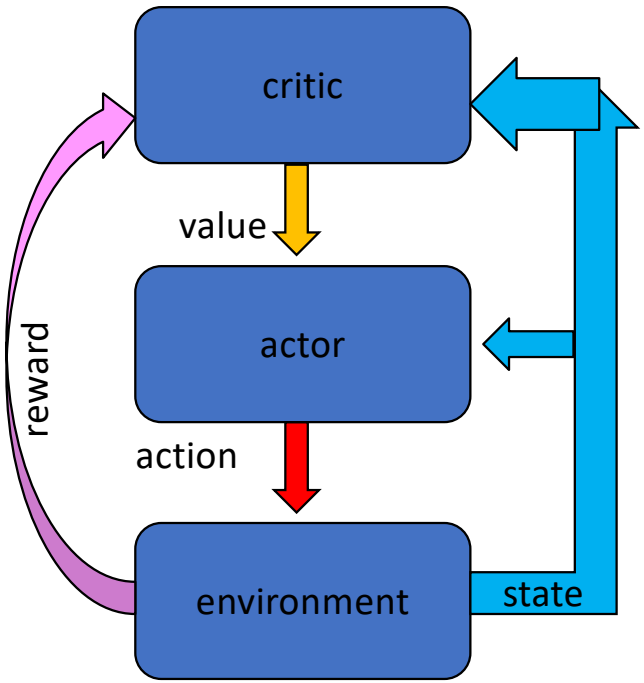
- Critic:
  - Input: state and reward
  - Goal: evaluate current policy
  - Method: TD
- Actor
  - Input: value and state
  - Goal: improve policy
  - Method: Near-greedy



# Actor-Critic Methodology

- Examples of Critic algorithms:
  - Monte Carlo
  - TD
  - Q-learning
- Example of Actor algorithms:
  - $\epsilon$-greedy
  - Soft-max
  - SARSA



# Part 3: outline

- Temporal Differences:
  - TD(0)
  - TD($\lambda$)
  - $SARSA(\lambda)$
- Importance Sampling
  - Behavioral policy
  - Evaluated policy
- Actor-Critic